

First-Order Logic According to Harrison

Alexander Birch Jensen, Anders Schlichtkrull &
Jørgen Villadsen, DTU Compute, Denmark

6 February 2026

Abstract

We present a certified declarative first-order prover with equality based on John Harrison's Handbook of Practical Logic and Automated Reasoning, Cambridge University Press, 2009. ML code reflection is used such that the entire prover can be executed within Isabelle as a very simple interactive proof assistant. As examples we consider Pelletier's problems 1-46.

Contents

Preamble	1
FOL-Harrison	2
Module Proven	2
ML Code Reflection	6
Main Examples	6
Other Examples	7
Acknowledgements	8

Preamble

Preliminary formalizations are described here:

Alexander Birch Jensen: *Development and Verification of a Proof Assistant*. Master's Thesis, Technical University of Denmark, 2016. <http://findit.dtu.dk/en/catalog/2345011633>

Alexander Birch Jensen, Anders Schlichtkrull & Jørgen Villadsen: *Verification of an LCF-Style First-Order Prover with Equality*. Isabelle Workshop 2016. <https://github.com/logic-tools/sml-handbook>

FOL-Harrison

```
theory FOL_Harrison
imports
  Main
begin
```

Module Proven

Syntax of first-order logic

```
type_synonym id = String.literal

datatype tm = Var id | Fn id "tm list"

datatype 'a fm = Truth | Falsity | Atom 'a | Imp "'a fm" "'a fm" | Iff "'a fm" "'a fm" |
  And "'a fm" "'a fm" | Or "'a fm" "'a fm" | Not "'a fm" | Exists id "'a fm" | Forall id "'a fm"

datatype fol = Rl id "tm list"

datatype "thm" = Thm (concl: "fol fm")
```

Definition of rules and axioms

```
abbreviation (input) "fail_thm  $\equiv$  Thm Truth"

definition fol_equal :: "fol fm  $\Rightarrow$  fol fm  $\Rightarrow$  bool"
where
  "fol_equal p q  $\equiv$  p = q"

definition zip_eq :: "tm list  $\Rightarrow$  tm list  $\Rightarrow$  fol fm list"
where
  "zip_eq l l'  $\equiv$  map ( $\lambda(t, t').$  Atom (Rl (STR ' '= ' ') [t, t'])) (zip l l')"

primrec occurs_in :: "id  $\Rightarrow$  tm  $\Rightarrow$  bool" and occurs_in_list :: "id  $\Rightarrow$  tm list  $\Rightarrow$  bool"
where
  "occurs_in i (Var x) = (i = x)" |
  "occurs_in i (Fn _ l) = occurs_in_list i l" |
  "occurs_in_list _ [] = False" |
  "occurs_in_list i (h # t) = (occurs_in i h  $\vee$  occurs_in_list i t)"

primrec free_in :: "id  $\Rightarrow$  fol fm  $\Rightarrow$  bool"
where
  "free_in _ Truth = False" |
  "free_in _ Falsity = False" |
  "free_in i (Atom a) = (case a of Rl _ l  $\Rightarrow$  occurs_in_list i l)" |
  "free_in i (Imp p q) = (free_in i p  $\vee$  free_in i q)" |
  "free_in i (Iff p q) = (free_in i p  $\vee$  free_in i q)" |
  "free_in i (And p q) = (free_in i p  $\vee$  free_in i q)" |
  "free_in i (Or p q) = (free_in i p  $\vee$  free_in i q)" |
  "free_in i (Not p) = free_in i p" |
  "free_in i (Exists x p) = (i  $\neq$  x  $\wedge$  free_in i p)" |
  "free_in i (Forall x p) = (i  $\neq$  x  $\wedge$  free_in i p)"

primrec equal_length :: "tm list  $\Rightarrow$  tm list  $\Rightarrow$  bool"
where
  "equal_length l [] = (case l of []  $\Rightarrow$  True | _ # _  $\Rightarrow$  False)" |
  "equal_length l (_ # r') = (case l of []  $\Rightarrow$  False | _ # l'  $\Rightarrow$  equal_length l' r')"

definition modusponens :: "thm  $\Rightarrow$  thm  $\Rightarrow$  thm"
```

```

where
  "modusponens s s'  $\equiv$  case concl s of Imp p q  $\Rightarrow$ 
    let p' = concl s' in if fol_equal p p' then Thm q else fail_thm | _  $\Rightarrow$  fail_thm"

definition gen :: "id  $\Rightarrow$  thm  $\Rightarrow$  thm"
where
  "gen x s  $\equiv$  Thm (Forall x (concl s))"

definition axiom_addimp :: "fol fm  $\Rightarrow$  fol fm  $\Rightarrow$  thm"
where
  "axiom_addimp p q  $\equiv$  Thm (Imp p (Imp q p))"

definition axiom_distribimp :: "fol fm  $\Rightarrow$  fol fm  $\Rightarrow$  fol fm  $\Rightarrow$  thm"
where
  "axiom_distribimp p q r  $\equiv$  Thm (Imp (Imp p (Imp q r)) (Imp (Imp p q) (Imp p r)))"

definition axiom_doubleneg :: "fol fm  $\Rightarrow$  thm"
where
  "axiom_doubleneg p  $\equiv$  Thm (Imp (Imp (Imp p Falsity) Falsity) p)"

definition axiom_allimp :: "id  $\Rightarrow$  fol fm  $\Rightarrow$  fol fm  $\Rightarrow$  thm"
where
  "axiom_allimp x p q  $\equiv$  Thm (Imp (Forall x (Imp p q)) (Imp (Forall x p) (Forall x q)))"

definition axiom_impall :: "id  $\Rightarrow$  fol fm  $\Rightarrow$  thm"
where
  "axiom_impall x p  $\equiv$  if  $\neg$  free_in x p then Thm (Imp p (Forall x p)) else fail_thm"

definition axiom_existseq :: "id  $\Rightarrow$  tm  $\Rightarrow$  thm"
where
  "axiom_existseq x t  $\equiv$  if  $\neg$  occurs_in x t
    then Thm (Exists x (Atom (Rl (STR ''='') [Var x, t]))) else fail_thm"

definition axiom_eqrefl :: "tm  $\Rightarrow$  thm"
where
  "axiom_eqrefl t  $\equiv$  Thm (Atom (Rl (STR ''='') [t, t]))"

definition axiom_funcong :: "id  $\Rightarrow$  tm list  $\Rightarrow$  tm list  $\Rightarrow$  thm"
where
  "axiom_funcong i l l'  $\equiv$  if equal_length l l'
    then Thm (foldr Imp (zip_eq l l') (Atom (Rl (STR ''='') [Fn i l, Fn i l']))) else fail_thm"

definition axiom_predcong :: "id  $\Rightarrow$  tm list  $\Rightarrow$  tm list  $\Rightarrow$  thm"
where
  "axiom_predcong i l l'  $\equiv$  if equal_length l l'
    then Thm (foldr Imp (zip_eq l l') (Imp (Atom (Rl i l)) (Atom (Rl i l')))) else fail_thm"

definition axiom_iffimp1 :: "fol fm  $\Rightarrow$  fol fm  $\Rightarrow$  thm"
where
  "axiom_iffimp1 p q  $\equiv$  Thm (Imp (Iff p q) (Imp p q))"

definition axiom_iffimp2 :: "fol fm  $\Rightarrow$  fol fm  $\Rightarrow$  thm"
where
  "axiom_iffimp2 p q  $\equiv$  Thm (Imp (Iff p q) (Imp q p))"

definition axiom_impiff :: "fol fm  $\Rightarrow$  fol fm  $\Rightarrow$  thm"
where
  "axiom_impiff p q  $\equiv$  Thm (Imp (Imp p q) (Imp (Imp q p) (Iff p q)))"

definition axiom_true :: "thm"
where
  "axiom_true  $\equiv$  Thm (Iff Truth (Imp Falsity Falsity))"

definition axiom_not :: "fol fm  $\Rightarrow$  thm"

```

```

where
  "axiom_not p  $\equiv$  Thm (Iff (Not p) (Imp p Falsity))"

definition axiom_and :: "fol fm  $\Rightarrow$  fol fm  $\Rightarrow$  thm"
where
  "axiom_and p q  $\equiv$  Thm (Iff (And p q) (Imp (Imp p (Imp q Falsity)) Falsity))"

definition axiom_or :: "fol fm  $\Rightarrow$  fol fm  $\Rightarrow$  thm"
where
  "axiom_or p q  $\equiv$  Thm (Iff (Or p q) (Not (And (Not p) (Not q))))"

definition axiom_exists :: "id  $\Rightarrow$  fol fm  $\Rightarrow$  thm"
where
  "axiom_exists x p  $\equiv$  Thm (Iff (Exists x p) (Not (Forall x (Not p))))"

```

Code generation for rules and axioms

```

export_code
  modusponens gen axiom_addimp axiom_distribimp axiom_doubleneg axiom_allimp axiom_impall
  axiom_existseq axiom_eqrefl axiom_funcong axiom_predcong axiom_iffimp1 axiom_iffimp2
  axiom_impiff axiom_true axiom_not axiom_and axiom_or axiom_exists concl
in SML module_name Proven

```

```

code_printing constant fol_equal  $\rightarrow$  (SML) "_ =_" — More efficient

```

```

export_code
  modusponens gen axiom_addimp axiom_distribimp axiom_doubleneg axiom_allimp axiom_impall
  axiom_existseq axiom_eqrefl axiom_funcong axiom_predcong axiom_iffimp1 axiom_iffimp2
  axiom_impiff axiom_true axiom_not axiom_and axiom_or axiom_exists concl
in SML module_name Proven

```

```

code_printing constant fol_equal  $\rightarrow$  (SML) — Delete

```

```

export_code
  modusponens gen axiom_addimp axiom_distribimp axiom_doubleneg axiom_allimp axiom_impall
  axiom_existseq axiom_eqrefl axiom_funcong axiom_predcong axiom_iffimp1 axiom_iffimp2
  axiom_impiff axiom_true axiom_not axiom_and axiom_or axiom_exists concl
in SML module_name Proven

```

Semantics of first-order logic

```

definition length2 :: "tm list  $\Rightarrow$  bool"
where
  "length2 l  $\equiv$  case l of [_,_]  $\Rightarrow$  True | _  $\Rightarrow$  False"

```

primrec — Semantics of terms

```

semantics_term :: "(id  $\Rightarrow$  'a)  $\Rightarrow$  (id  $\Rightarrow$  'a list  $\Rightarrow$  'a)  $\Rightarrow$  tm  $\Rightarrow$  'a" and
semantics_list :: "(id  $\Rightarrow$  'a)  $\Rightarrow$  (id  $\Rightarrow$  'a list  $\Rightarrow$  'a)  $\Rightarrow$  tm list  $\Rightarrow$  'a list"
where
  "semantics_term e _ (Var x) = e x" |
  "semantics_term e f (Fn i l) = f i (semantics_list e f l)" |
  "semantics_list _ _ [] = []" |
  "semantics_list e f (t # l) = semantics_term e f t # semantics_list e f l"

```

primrec — Semantics of formulas

```

semantics :: "(id  $\Rightarrow$  'a)  $\Rightarrow$  (id  $\Rightarrow$  'a list  $\Rightarrow$  'a)  $\Rightarrow$  (id  $\Rightarrow$  'a list  $\Rightarrow$  bool)  $\Rightarrow$  fol fm  $\Rightarrow$  bool"
where
  "semantics _ _ _ Truth = True" |
  "semantics _ _ _ Falsity = False" |
  "semantics e f g (Atom a) = (case a of Rl i l  $\Rightarrow$  if i = STR ''='''  $\wedge$  length2 l
    then (semantics_term e f (hd l) = semantics_term e f (hd (tl l)))
    else g i (semantics_list e f l))" |
  "semantics e f g (Imp p q) = (semantics e f g p  $\longrightarrow$  semantics e f g q)" |

```

```

"semantics e f g (Iff p q) = (semantics e f g p  $\longleftrightarrow$  semantics e f g q)" |
"semantics e f g (And p q) = (semantics e f g p  $\wedge$  semantics e f g q)" |
"semantics e f g (Or p q) = (semantics e f g p  $\vee$  semantics e f g q)" |
"semantics e f g (Not p) = ( $\neg$  semantics e f g p)" |
"semantics e f g (Exists x p) = ( $\exists v.$  semantics (e(x := v)) f g p)" |
"semantics e f g (Forall x p) = ( $\forall v.$  semantics (e(x := v)) f g p)"

```

Definition of proof system

```

inductive OK :: "fol fm  $\Rightarrow$  bool" (<|_> 0)
where
  modusponens:
    "|_ concl s  $\Rightarrow$  |_ concl s'  $\Rightarrow$  |_ concl (modusponens s s')" |
  gen:
    "|_ concl s  $\Rightarrow$  |_ concl (gen _ s)" |
  axiom_addimp:
    "|_ concl (axiom_addimp _ _)" |
  axiom_distribimp:
    "|_ concl (axiom_distribimp _ _ _)" |
  axiom_doubleneg:
    "|_ concl (axiom_doubleneg _)" |
  axiom_allimp:
    "|_ concl (axiom_allimp _ _ _)" |
  axiom_impall:
    "|_ concl (axiom_impall _ _)" |
  axiom_existseq:
    "|_ concl (axiom_existseq _ _)" |
  axiom_eqrefl:
    "|_ concl (axiom_eqrefl _)" |
  axiom_funcong:
    "|_ concl (axiom_funcong _ _ _)" |
  axiom_predcong:
    "|_ concl (axiom_predcong _ _ _)" |
  axiom_iffimp1:
    "|_ concl (axiom_iffimp1 _ _)" |
  axiom_iffimp2:
    "|_ concl (axiom_iffimp2 _ _)" |
  axiom_impiff:
    "|_ concl (axiom_impiff _ _)" |
  axiom_true:
    "|_ concl axiom_true" |
  axiom_not:
    "|_ concl (axiom_not _)" |
  axiom_and:
    "|_ concl (axiom_and _ _)" |
  axiom_or:
    "|_ concl (axiom_or _ _)" |
  axiom_exists:
    "|_ concl (axiom_exists _ _)"

```

proposition "|_ Imp p p"
<proof>

Soundness of proof system

```

lemma map':
  " $\neg$  occurs_in x t  $\Rightarrow$  semantics_term e f t = semantics_term (e(x := v)) f t"
  " $\neg$  occurs_in_list x l  $\Rightarrow$  semantics_list e f l = semantics_list (e(x := v)) f l"
<proof>

```

lemma map:

```

"¬ free_in x p ⇒ semantics e f g p ⇔ semantics (e(x := v)) f g p"
⟨proof⟩

lemma length2_equiv:
  "length2 l ⇔ [hd l, hd (tl l)] = l"
⟨proof⟩

lemma equal_length_sym:
  "equal_length l l' ⇒ equal_length l' l"
⟨proof⟩

lemma equal_length2:
  "equal_length l l' ⇒ length2 l ⇔ length2 l'"
⟨proof⟩

lemma imp_chain_equiv:
  "semantics e f g (foldr Imp l p) ⇔ (∀q ∈ set l. semantics e f g q) → semantics e f g p"
⟨proof⟩

lemma imp_chain_zip_eq:
  "equal_length l l' ⇒
    semantics e f g (foldr Imp (zip_eq l l') p) ⇔
    semantics_list e f l = semantics_list e f l' → semantics e f g p"
⟨proof⟩

lemma funcong:
  "equal_length l l' ⇒
    semantics e f g (foldr Imp (zip_eq l l') (Atom (Rl (STR ''=''') [Fn i l, Fn i l'])))"
⟨proof⟩

lemma predcong:
  "equal_length l l' ⇒
    semantics e f g (foldr Imp (zip_eq l l') (Imp (Atom (Rl i l)) (Atom (Rl i l'))))"
⟨proof⟩

theorem soundness:
  "⊢ p ⇒ semantics e f g p"
⟨proof⟩

corollary "¬ (⊢ Falsity)"
⟨proof⟩

```

ML Code Reflection

```

code_reflect
  Proven
datatypes
  fm = Falsity | Truth | Atom | Imp | Iff | And | Or | Not | Exists | Forall
and
  tm = Var | Fn
and
  fol = Rl
functions
  modusponens gen axiom_addimp axiom_distribimp axiom_doubleneg axiom_allimp axiom_impall
  axiom_existseq axiom_eqrefl axiom_funcong axiom_predcong axiom_iffimp1 axiom_iffimp2
  axiom_impiff axiom_true axiom_not axiom_and axiom_or axiom_exists concl
⟨ML⟩

```

Main Examples

⟨ML⟩

Other Examples

$\langle ML \rangle$

end

Acknowledgements

The SML code is based on the OCaml code accompanying John Harrison's Handbook of Practical Logic and Automated Reasoning, Cambridge University Press, 2009

Thanks to Jasmin Blanchette, Asta Halkjær From, John Bruntse Larsen, Andrei Popescu and Tom Ridge for discussions.